

4. Solve 8-Puzzle using A* Search and analyse

```
import heapq
```

```
def print_board(state):
```

```
    for i in range(0, 9, 3):
```

```
        print(state[i:i+3])
```

```
    print()
```

```
def get_neighbors(state):
```

```
    neighbors = []
```

```
    zero = state.index(0)
```

```
    moves = {
```

```
        0: [1, 3],
```

```
        1: [0, 2, 4],
```

```
        2: [1, 5],
```

```
        3: [0, 4, 6],
```

```
        4: [1, 3, 5, 7],
```

```
        5: [2, 4, 8],
```

```
        6: [3, 7],
```

```
        7: [4, 6, 8],
```

```
8: [5, 7]
}
```

```
for pos in moves[zero]:
```

```
    new_state = state.copy()
```

```
    new_state[zero], new_state[pos] = \
        new_state[pos], new_state[zero]
```

```
    neighbors.append(new_state)
```

```
return neighbors
```

```
def heuristic(state, goal):
```

```
    distance = 0
```

```
    for i in range(9):
```

```
        if state[i] == 0:
            continue
```

```
        row = i // 3
```

```
col = i % 3
```

```
goal_index = goal.index(state[i])
```

```
goal_row = goal_index // 3
```

```
goal_col = goal_index % 3
```

```
distance += abs(row - goal_row)
```

```
distance += abs(col - goal_col)
```

```
return distance
```

```
def a_star(start, goal):
```

```
    priority_queue = []
```

```
    g = 0
```

```
    h = heuristic(start, goal)
```

```
    f = g + h
```

```
    heapq.heappush(
```

```
        priority_queue,
```

```
        (f, g, start, [start])
```

```
    )
```

```
visited = set()
```

```
while priority_queue:
```

```
    f, g, state, path = heapq.heappop(priority_queue)
```

```
    if tuple(state) in visited:
```

```
        continue
```

```
    visited.add(tuple(state))
```

```
    if state == goal:
```

```
        return path
```

```
    for next_state in get_neighbors(state):
```

```
        if tuple(next_state) not in visited:
```

```
            new_g = g + 1
```

```
            new_h = heuristic(next_state, goal)
```

```
            new_f = new_g + new_h
```

```
            heapq.heappush(
```

```
        priority_queue,  
        (  
            new_f,  
            new_g,  
            next_state,  
            path + [next_state]  
        )  
    )  
)
```

```
return None
```

```
start = [1, 2, 3,  
         4, 0, 6,  
         7, 5, 8]
```

```
goal = [1, 2, 3,  
        4, 5, 6,  
        7, 8, 0]
```

```
solution = a_star(start, goal)
```

```
if solution:
```

```
    print("Solution using A* Search")
```

```
print("Number of moves:", len(solution) - 1)
```

```
for state in solution:
```

```
    print_board(state)
```

```
else:
```

```
    print("No solution found")
```

A* is one of the most important heuristic search algorithms.

It uses:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = cost from start to current state
- $h(n)$ = estimated cost from current state to goal
- $f(n)$ = total estimated cost